

Pointers

Introduction to Pointers in C

Address	Contents
1000	0010 0011
ptr → 1001	0101 0011
1002	0001 0010
1003	0010 1000
1004	0100 1100
	⋮
1020	0011 0001
1021	

1 byte
1 byte = 8 bits

a

int a;

* Pointer is a special variable that is capable of storing some address.



* it points to a memory location where the first byte is stored

Syntax for Declaring Pointer Variables

General Syntax for declaring pointer variables

data-type * pointer_name;

Example:

int *ptr; ←

char *ptr; ←

float *ptr; ←

Points to integer Value

Points to character Value

Points to float Value

Initialize pointer

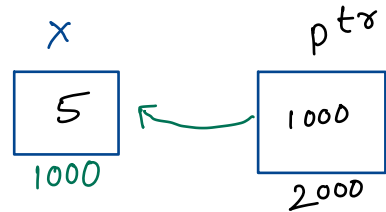
One way to initialize a Pointer is to assign address of some Variable.

```
int x = 5;
```

```
int *ptr;
```

```
ptr = &x
```

↑ address of operator



```
int x = 5, *ptr = &x;
```

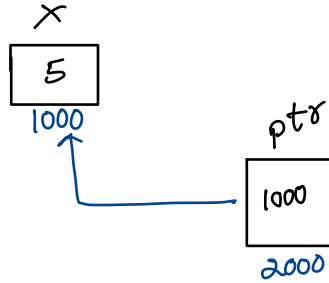
```
char c = 'A', *ptr = &c;
```

```
float f = 2.5, *ptr = &f;
```

* Value of operator

Value of the operator/indirection operator/dereference operator is an operator that is used to access the value stored at the location pointed by the Pointer.

```
int x = 5;  
int *ptr = &x;  
printf("%d", *ptr);
```

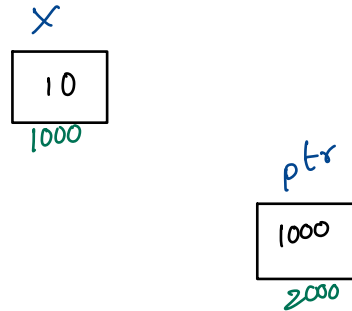


Value of operator

* We can also change the Value of the object pointed by the Pointer.

Example:

```
int x = 10;  
int *ptr = &x;  
*ptr = 4;
```



! Never apply the indirection operator to the Uninitialized Pointer

Example:

```
int *ptr;  
printf("%d", *ptr);
```

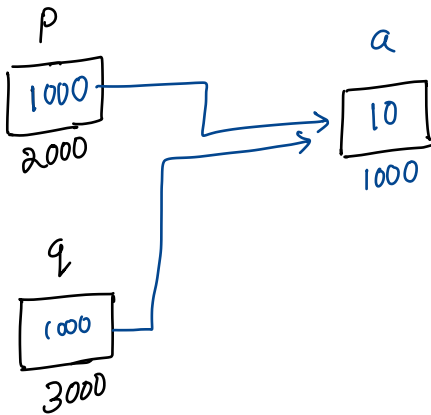
! Assigning Value to an Uninitialized pointer is

Dangerous.

```
int *ptr;  
*ptr = 1;
```

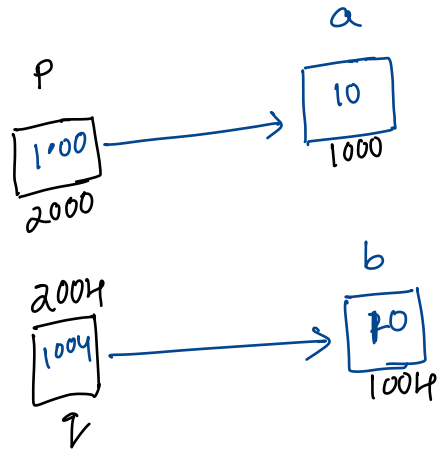
Pointer Assignment

```
int a = 10;  
int *p, *q;  
p = &a;  
q = p;  
printf("%d %d", *p, *q);
```



Note :- $q = p$ is not Same as

```
*q = *p  
int a = 10, b = 20;  
int *p, *q;  
p = &a;  
q = &b;  
*q = *p;
```



```

#include <stdio.h>
#include<stdlib.h>
#include<math.h>
#define INT_MIN -2147483648

int main(void){
    //pointers
    //Data_type *name_pointer;
    int *ptr; // declaring a pointer
    char *ptr1;
    float *ptr2;
    //initilization pointers using address of operator
    int x = 5;
    ptr = &x;
    char c = 'A';
    ptr1 = &c;
    float f = 2.5;
    ptr2 = &f;
    //value of operator
    //printf("%d\n",*ptr);
    *ptr = 10;
    //printf("%d",x);
    //pointer Assignment
    int a = 10;
    int b = 20;
    int *p , *q;
    p = &a;
    q = &b;
    *q = *p;
    printf("%d %d",p,q);
}

```

```

#include <stdio.h>
#include<stdlib.h>
#include<math.h>
#define INT_MIN -2147483648
//scope of variables
//local , global
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void){
    int x = 10;
    int y = 20;
    swap(&x,&y);
    printf("%d %d",x,y);
}

```

Swap

```

#include <stdio.h>
#include<stdlib.h>
#include<math.h>
#define INT_MIN -2147483648
//scope of variables
//local , global
void MinMax(int A[],int n, int *mini, int *maxi){
    for(int i = 1 ; i < n ; i++){
        if(*maxi < A[i]){
            *maxi = A[i];
        }
        if(*mini > A[i]){
            *mini = A[i];
        }
    }
}

int main(void){
    int A[] = {20,30,10,-1,-7,21,101};
    //i want to find max and min in array A
    //max = 101 min = -7
    int n = sizeof(A)/sizeof(int);
    int mini = A[0];
    int maxi = A[0];
    MinMax(A,n,&mini,&maxi);
    printf("%d %d",mini,maxi);
}

```

Basics of Dynamic memory Allocation

Static memory Allocation

- ⇒ memory allocated during Compile time is called Static memory.
- ⇒ The memory allocated is fixed and cannot be increased or decreased during run time.

Example:

```
int main()
{
    int A[5] = {-1, 10, 2, 0, 7};
}
```

Problems faced in static memory Allocation

! if you are allocating memory for an array during Compile time then you have to fix the Size at the time of declaration. Size is fixed and User can not increase or decrease the Size of the array at run time.

! if the values stored by the User in the array at run time is less than the Size specified then there will be wastage of memory.

! if the values stored by the user in the array at run time is more than the size specified then the program may crash or misbehave.

Dynamic memory Allocation

⇒ The process of allocating memory at the time of execution is called **Dynamic memory allocation**

⇒ There are certain built-in functions that can help in allocating or deallocating some memory space at run time.

Note:

⇒ Pointers play an important role in Dynamic memory Allocation

⇒ Allocated memory can only be accessed through pointers.

Built in functions

★ `malloc()`

★ `calloc()`

★ `realloc()`

★ `free()`

* malloc ()

malloc is a built-in function declared in the header

file `<stdlib.h>`

malloc is the short name for "memory allocation" and is used to dynamically allocate a single large block of contiguous memory according to the size specified

Syntax-

`(void *) malloc (size_t size)`

`int A[] = {20, 30, 10, -1, -7, 21, 101};`

101
maximum & minimum

`int maxi = 20;`

`int mini = 20;`

`for (i = 1 ; i < n ; i++) {`

`if (maxi < A[i])
maxi = A[i];`

`if (mini > A[i])
mini = A[i];`

`}`